Chapter 1

QELM Architecture and Implementation

1.1 The Hybrid Architecture

L'approccio ibrido utilizzato per l'implementazione di un algoritmo di classificazione, tramite il paradigma del QELM che ho adottato è il seguente: Scelta del dataset (MNIST cifre 28x28 pixel) -¿ Autoencoder (riduzione delle feature) -¿ Layer quantistico (Reservoir) -¿ Classificatore finale (ridge regression)

Avendo scelto il dataset MNIST, ci troviamo difronte a immagini composte da 784 pixel ciascuna. E' indispensabile a questo punto notare che siamo difronte al NISQ bottleneck in particolare su simulatore quantistico tramite hardware classico. Diventa necessario quindi implmenetare meccanismi che ci permettano di estrarre le giuste feature in modo da poter rappresentare ciascun immagine attraverso il suo spazio latente.

1.2 Addressing the NISQ bottleneck

Il decoder risulta fondamentale per l'apprendimento dell'AE, in quanto viene confrontata l'immagine orginale x con quella ricostruita dal decoder x', durante il loop di training, lavorando sulla minimizzazione del MSE.

self.fc$_decoder = nn.Linear(latent_dim, self.flatten_dim)$

self.decoder = nn.Sequential(Input (Batch, 32, 7, 7) -¿ Output : (Batch, 16, 14, 14) nn.ConvTranspose2d(32,16,kernel$_size = 3, stride = 2, padding = 1, output_padding = 1), nn.BatchNorm2d(16), nn.ReLU()$,

Input (Batch, 16, 14, 14) -¿ Output (Batch, 1, 28,28) nn.ConvTranspose2d(16,1,kernel$_size = 3, stride = 2, padding = 1, output_padding = 1)$,

nn.Sigmoid())

Da notare come in sostanza stiamo riespandendo le immagini, con delle accoretezze come "output$_padding = 1"percorreggereeventualiasimmetrieVediamolaSigmoidfinaleinquantoleim$